

TransBot システム設計書

多言語対応・GUI統合型 Discord 翻訳ボット アーキテクチャおよび機能仕様

1. システム概要 (System Overview)

TransBotは、Discordサーバー向けに設計された高度な多言語翻訳ボットです。Tkinterによる充実したローカルGUIインターフェースと、CLIのみで動作するヘッドレスモードの両方をサポートしています。Google Translate APIとGemini APIを統合し、リアルタイムのチャンネル間相互翻訳、AIによる推敲、ダイレクトメッセージ(DM)の管理、スパム対策機能を提供します。

特徴: 133言語をサポートし、Webhookを利用した自然なアバター偽装送信、またはEmbedによる軽量の翻訳表示を柔軟に切り替え可能です。

2. 技術スタック・利用ライブラリ (Technology Stack)

- 言語・フレームワーク: Python 3.x, `discord.py` (Discord API通信)
- GUI: `tkinter` (標準ライブラリ), `PIL (Pillow)` (画像処理・プレビュー表示)
- 翻訳エンジン: `deep_translator (GoogleTranslator)`, `google-genai` (Gemini 3.5 Flashモデル)
- データベース・暗号化: `sqlite3` (翻訳キャッシュ), `cryptography.fernet` (Token・API Keyの暗号化保存)
- 非同期処理: `asyncio`, `threading` (UIとDiscordループの分離)

3. システムアーキテクチャ (Architecture)

3.1. データフローとコンポーネント

システムは大きく「GUIプロセス(メインスレッド)」と「Discord Botプロセス(非同期ループ)」に分かれています。

- MultiServerApp (GUI):** ユーザーインターフェースを管理。各種設定、翻訳フィードの表示、ログの閲覧、メッセージの手動送信を行う。

- **TranslatorBot (Discord Client):** Discordからのイベント(メッセージ受信、リアクション、スラッシュコマンド)をリッスンし、キュー(`asyncio.Queue`)を利用して順次処理・送信を行う。
- **TranslationCache:** SQLiteとメモリ内LRUキャッシュのハイブリッド構造。APIコールの削減とパフォーマンス向上を担う。

3.2. ストレージ・ファイル構成

ファイル名	役割・用途	形式
<code>multi_server_config.json</code>	サーバーごとの翻訳グループ、通知設定、スパム閾値などの構成情報。トークン類は暗号化されて保存される。	JSON
<code>secret.key</code>	Fernet暗号化キー。Bot TokenとGemini APIキーの暗号化/復号化に使用。	Binary
<code>translation_cache.db</code>	翻訳結果を保存するSQLiteデータベース。キーは <code>(source_text, target_lang)</code> 。	SQLite3
<code>dm_history.json</code>	Botに送信されたダイレクトメッセージ(DM)の履歴を保存するファイル。	JSON
<code>lang_constants.py</code>	133言語のマスタデータ(日本語/英語)、UIローカライズテキスト、国旗絵文字とシステム言語の紐付け定義。	Python Dict

4. 機能仕様 (Functional Specifications)

4.1. コア翻訳機能

- **自動チャンネル同期:** 事前に設定されたグループ内のチャンネルにメッセージが投稿されると、他のチャンネルの指定言語に自動翻訳して配信。
- **Webhook / Embed 切り替え:** Webhookが有効な場合、発言者の名前とアイコンを偽装して自然に翻訳を表示。無効な場合はEmbed(埋め込み)形式で表示。
- **AI翻訳・推敲モード:** Gemini APIを利用し、単純な機械翻訳ではなく文脈に沿った自然な翻訳や文章の推敲を行う(`/ai` コマンド、GUIからのAI送信モード)。
- **国旗リアクション翻訳:** メッセージに国旗絵文字でリアクションすると、その言語に翻訳した結果をBotがリプライする。

4.2. モデレーションとセキュリティ機能

- **スパム検知:** 設定された時間枠(分)とメッセージ数の閾値を超えた連続投稿を検知し、自動的に一時停止 (Suspend) リストに登録。
- **BANシステム:** GUIまたは `/botconfig ban` コマンドにより、特定のユーザーやサーバーからのメッセージ処理を完全に無視。
- **Token暗号化保存:** 機密情報であるBot TokenとAPI Keyは `cryptography` を用いてローカルストレージ内で暗号化。

4.3. GUI モジュール機能

- **コントロールパネル:** メイン画面で接続中の全サーバー・チャンネル設定を階層ツリー表示。Webhook/国旗翻訳のトグルが可能。
- **DM Viewer:** ユーザーからのDMをGUI上で閲覧し、翻訳機能を利用しながら返信できるインターフェース。
- **Log Viewer:** 特定のチャンネルの過去ログを取得し、GUI上で一覧表示。画像アタッチメントのインライン表示および原寸拡大に対応。
- **自動多言語チャンネル生成 (Auto Clone):** ベースとなるチャンネルを複製し、選択した複数の言語用チャンネルを一括作成する機能。

5. コマンド設計 (Slash Commands)

Discord側から操作可能なスラッシュコマンドは以下の通りです。

コマンド	権限	機能概要
<code>/chset</code> <code>[count]</code>	サーバー 管理者	指定した数のチャンネルを選択し、新しい相互翻訳グループを構成する。
<code>/chdel</code> <code>[channel]</code>	サーバー 管理者	指定したチャンネルが所属する翻訳グループを削除する。
<code>/setup</code>	サーバー 管理者	現在のチャンネルの言語、Webhook、リアクション翻訳などの状態を確認・変更できるインタラクティブなUI(コントロールパネル)を表示。
<code>/chadd [lang]</code>	サーバー 管理者	既存の翻訳グループに新しいチャンネルを追加・接続する。
<code>/chmode [wh]</code> <code>[re]</code>	サーバー 管理者	Webhook機能、国旗リアクション機能のON/OFFを切り替える。

コマンド	権限	機能概要
<code>/ai [text]</code>	全ユーザー	Gemini APIを使用して入力テキストを推敲・翻訳し、グループ内に配信する。
<code>/botconfig [...]</code>	システム管理者	ボット全体のステータス確認、スパム閾値変更、通知設定、ユーザーBANの管理を行うサブコマンド群。

6. 内部モジュールとクラス設計 (Class Design)

6.1. TranslationCache (翻訳キャッシュ機構)

外部APIのレート制限と遅延を回避するため、SQLiteと `collections.OrderedDict` を用いた二段構えのキャッシュを採用。最大メモリサイズ(デフォルト5000)を超えた場合はLRUアルゴリズムで破棄されるが、DBには永続化される。

6.2. TranslatorBot (Discord クライアント)

`discord.Client` を継承。以下のイベントコールバックをオーバーライドして実装：

- `setup_hook` : コマンドツリーの同期、メッセージ送信キュー(`msg_queue`)を処理するバックグラウンドタスクの起動。
- `on_message` : メッセージのルーティング、スパム判定、DMのインターセプト、グループ内他チャンネルへのブロードキャスト(`translate_and_queue`)を処理。
- `on_raw_reaction_add` : 国旗絵文字(`lang_constants.py` の `FLAG_TO_LANG`)を検知した場合のリプライ翻訳処理。

6.3. GUI ウィンドウクラス (Tkinter)

- `MultiServerApp` (メイン): ウィンドウのルート。ツリービューによる設定の可視化とシステムログの表示を行う。
- `SettingsWindow` : UI言語、APIキー、ログ件数、スパム設定、BANリストを管理。
- `DMViewerWindow` : ユーザーからのDMを一覧・履歴表示し、「そのまま送信」「翻訳して送信」を選択できる。
- `LogViewerWindow` : チャンネルのメッセージ履歴(`channel.history`)を取得・表示し、一括翻訳機能を提供する。
- `MessageSenderWindow` : GUIから特定のチャンネルへBotとしてメッセージ(テキスト/AI生成)を送信する。